ART GALLERY MANAGEMENT

PROJECT REPORT

Project Code: CPP64

Enrollment Number: IU2541230539

Programming Language: C++17

Prepared: October 12, 2026

Submission deadline supplied: October 13, 2026

Table of Contents

- 1. Project Title and Student Details
- 2. Problem Statement
- 3. Objectives
- 4. Features and Functional Requirements
- 5. Classes and Responsibilities
- 6. Object-Oriented Programming Concepts
- 7. Data Model and File Handling
- 8. System Workflow
- 9. UML Diagrams
- 10. User Interface and Screenshots
- 11. Testing and Validation
- 12. Challenges and Solutions
- 13. Results and Discussion
- 14. Limitations and Future Scope
- 15. Conclusion
- 16. References and Source Basis
- 17. Appendix: Viva / Run Instructions

1. Project Title and Student Details

Project Title	Art Gallery Management
Project Code	CPP64
Enrollment Number	IU2541230539
Programming Language	C++17
Application Type	Console-based Object-Oriented Management System
GitHub Repository	[PASTE ACTUAL GITHUB REPOSITORY LINK HERE, IF REQUIRED]

This final report represents a unified implementation created by evaluating and consolidating the two supplied project packages. The stronger technical architecture, validation and evidence from the first package were retained, while useful domain fields and presentation details from the second package were integrated.

2. Problem Statement

A small art gallery needs a consistent way to maintain artwork, artist, customer and exhibition records. Manual registers can lead to duplicate identifiers, incomplete information, difficulty locating records, accidental resale of sold artwork and loss of data after the application closes. The proposed system centralizes these records in a menu-driven C++ application, validates relationships between records, processes controlled artwork sales, generates management reports and stores a recoverable local snapshot.

The project is intentionally scoped as a single-operator academic console application. It does not claim to be a production payment system, multi-user database or online marketplace.

3. Objectives

- Maintain artwork, artist, customer and exhibition records through one menu-driven application.
- Provide add, display, search, update and delete operations.
- Link each artwork to an existing artist and protect referenced records from destructive deletion.
- Process an artwork sale only when the artwork exists, is available and the customer exists.
- Record transaction amount and timestamp and prevent the same artwork from being sold twice.
- Demonstrate encapsulation, constructors/destructors, inheritance, runtime polymorphism, dynamic memory, exception handling and file handling.
- Provide reliable save/load behavior and verify persistence after restarting the application.
- Generate a clear report showing inventory, availability and sales revenue.

4. Features and Functional Requirements

Function	Description
Add Record	Creates Artwork, Artist, Customer or Exhibition records with validated positive IDs and required fields.
Display Records	Displays the selected category; Artist and Customer demonstrate polymorphic display through Person.
Search Record	Performs exact positive-ID lookup and reports when a record does not exist.
Update Record	Updates descriptive information while retaining the ID. Sold artwork is locked.
Delete Record	Deletes safe records while blocking deletion that would break artwork or transaction references.
Sell Artwork	Checks artwork availability and customer existence, asks for YES confirmation, records a timestamped sale and

	saves the snapshot.
Report	Shows counts, available/sold artworks, available inventory value, revenue and transaction history.
Save/Load	Explicit save or confirmed reload. Reload builds and validates a temporary state before replacing the live state.
Exit	Saves the current state and closes the program.

5. Classes and Responsibilities

Class	Responsibility
Person	Abstract base class containing ID, name and contact information; declares virtual display().
Artist	Inherits Person and stores specialization/specialty information.
Customer	Inherits Person and stores address information.
Artwork	Stores ID, title, artist ID, category, price in paise and sold status.
Exhibition	Stores ID, name, start date, end date and theme.
Transaction	Stores transaction ID, artwork ID, customer ID, amount and timestamp.
Gallery	Owns records, performs CRUD operations, sales, reports, validation and dynamic artwork-array growth.
FileManager	Serializes/deserializes the Gallery state using a versioned local snapshot.

6. Object-Oriented Programming Concepts

- Classes and objects: real gallery entities are modeled as C++ classes.
- Encapsulation: data members are private/protected and accessed through controlled methods.
- Inheritance: Artist and Customer inherit common Person attributes.
- Runtime polymorphism: Person declares a pure virtual display(); Gallery invokes the appropriate override through a Person reference.
- Constructors and destructors: each domain class provides construction and cleanup behavior.
- Dynamic memory: Gallery manages a dynamically allocated Artwork array and grows it safely when capacity is reached.
- Exception handling: invalid input, missing records and malformed files are reported with exceptions.
- File handling: FileManager persists the complete application state and restores it after restart.

7. Data Model and File Handling

The merged implementation uses the robust versioned snapshot design from the stronger project while incorporating the richer domain fields of the second project. Data/gallery.txt begins with ART_GALLERY_V2 and contains ordered sections for artists, customers, exhibitions, artworks and transactions.

7.1 Record Fields

- Artwork: ID | Title | Artist ID | Category | Price (paise) | Sold flag
- Artist: ID | Name | Contact | Specialty
- Customer: ID | Name | Contact | Address
- Exhibition: ID | Name | Start Date | End Date | Theme
- Transaction: ID | Artwork ID | Customer ID | Amount (paise) | Timestamp

7.2 Persistence Safety

- State is validated before saving.
- Save first writes a temporary file and checks the write result.
- The previous snapshot is retained as gallery.bak before replacement.
- Load constructs a temporary Gallery, validates relationships and transaction consistency, then swaps it into the live object.
- Version 1 data from the original project remains readable with sensible default values for the newly added fields.

8. System Workflow

- Start the application and resolve the data directory.
- Load and validate the existing snapshot, or start empty if no data exists.
- Display the nine-option menu.
- Read and validate the user's selected operation.
- Perform the selected CRUD, sale, report or persistence operation.
- For a sale, verify artwork and customer, request YES confirmation, create the transaction, mark the artwork SOLD and save.
- Generate reports from the current state.
- Save before exit.

9. UML Diagrams

The package contains Use Case, Class and Sequence diagrams. The diagrams from the stronger project were retained because they clearly communicate the system architecture and interaction flow.

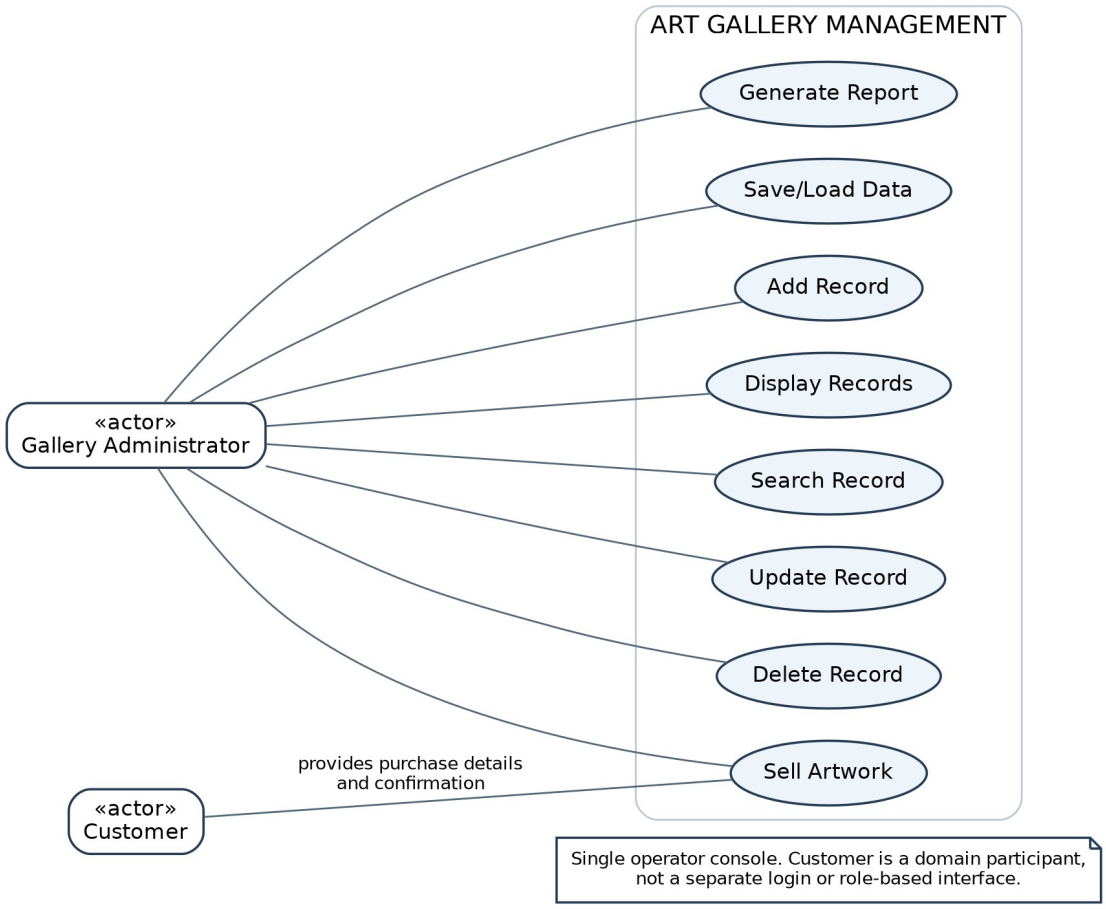


Figure 1. Use Case Diagram

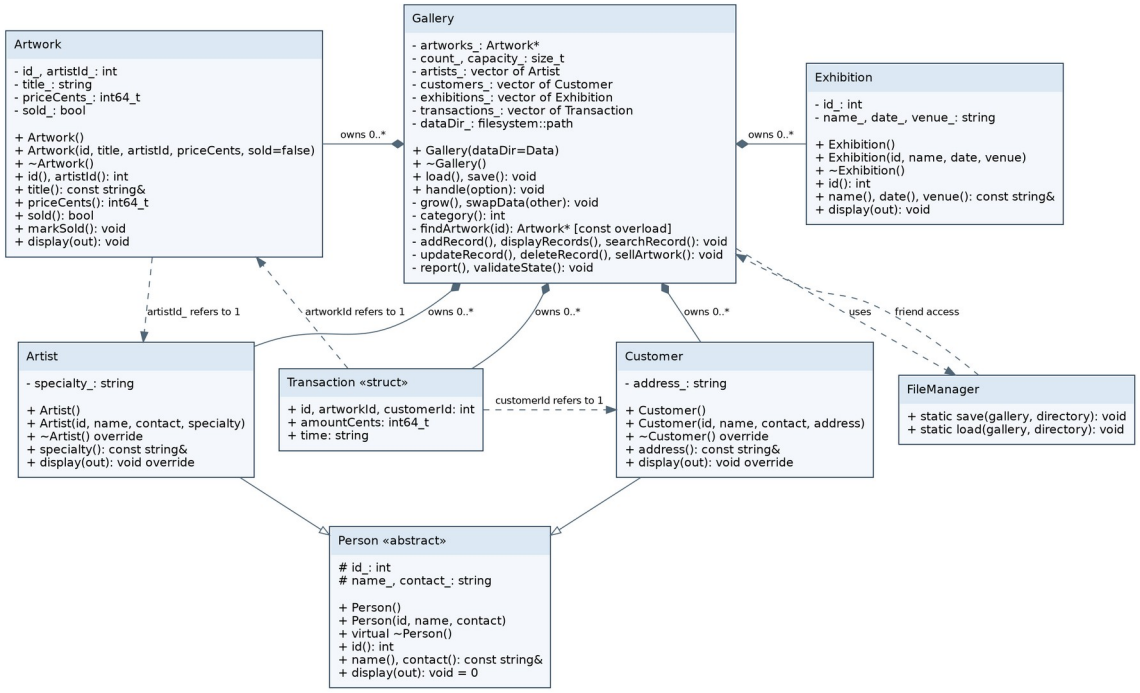


Figure 2. Class Diagram

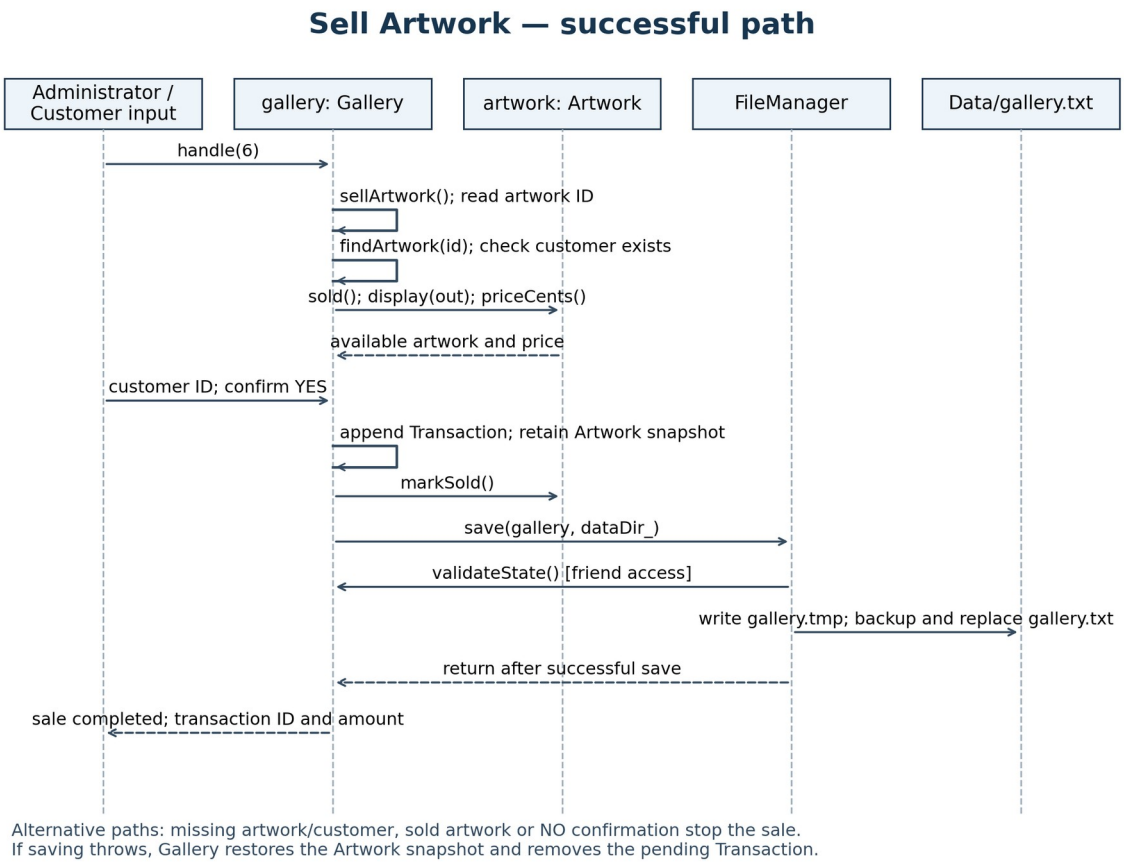


Figure 3. Sequence Diagram — Record Operation

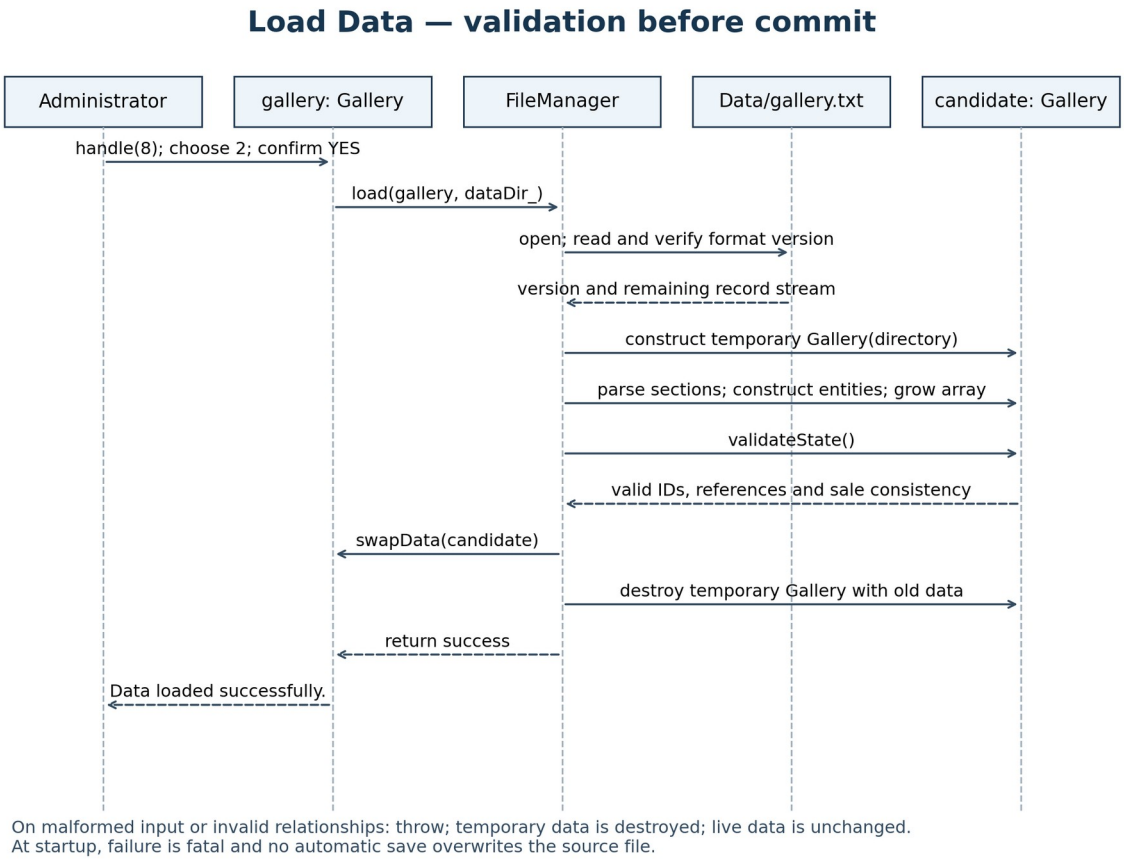


Figure 4. Sequence Diagram — Artwork Sale

10. User Interface and Screenshots

The supplied package contains executable-output screenshots covering the main menu, add, display, search, update, delete, transaction, report, save, load and persistence flows. They are retained as supporting visual evidence. Because the merged version adds category and richer exhibition fields, the student should run the final Windows build locally and capture any updated screens needed for the final submission.

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

01_MainMenu.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Record type (1 Artwork, 2 Artist, 3 Customer, 4 Exhibition): ID: Title: Artist ID: Price (INR): Record a
dded successfully. Save using option 8 or Exit.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

02_AddRecord.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Record type (1 Artwork, 2 Artist, 3 Customer, 4 Exhibition): Artwork #101 | Monsoon Valley | Artist #1 |
 INR 25000.00 | AVAILABLE
Artwork #102 | City in Motion | Artist #2 | INR 32000.00 | AVAILABLE
Artwork #103 | Quiet Lake | Artist #3 | INR 18500.00 | AVAILABLE
Artwork #104 | Golden Horizon | Artist #1 | INR 41000.00 | AVAILABLE
Artwork #105 | Rhythm of Light | Artist #2 | INR 27500.00 | AVAILABLE

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

03_DisplayRecords.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Record type (1 Artwork, 2 Artist, 3 Customer, 4 Exhibition): ID: Artwork #101 | Monsoon Valley | Artist
#1 | INR 25000.00 | AVAILABLE

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

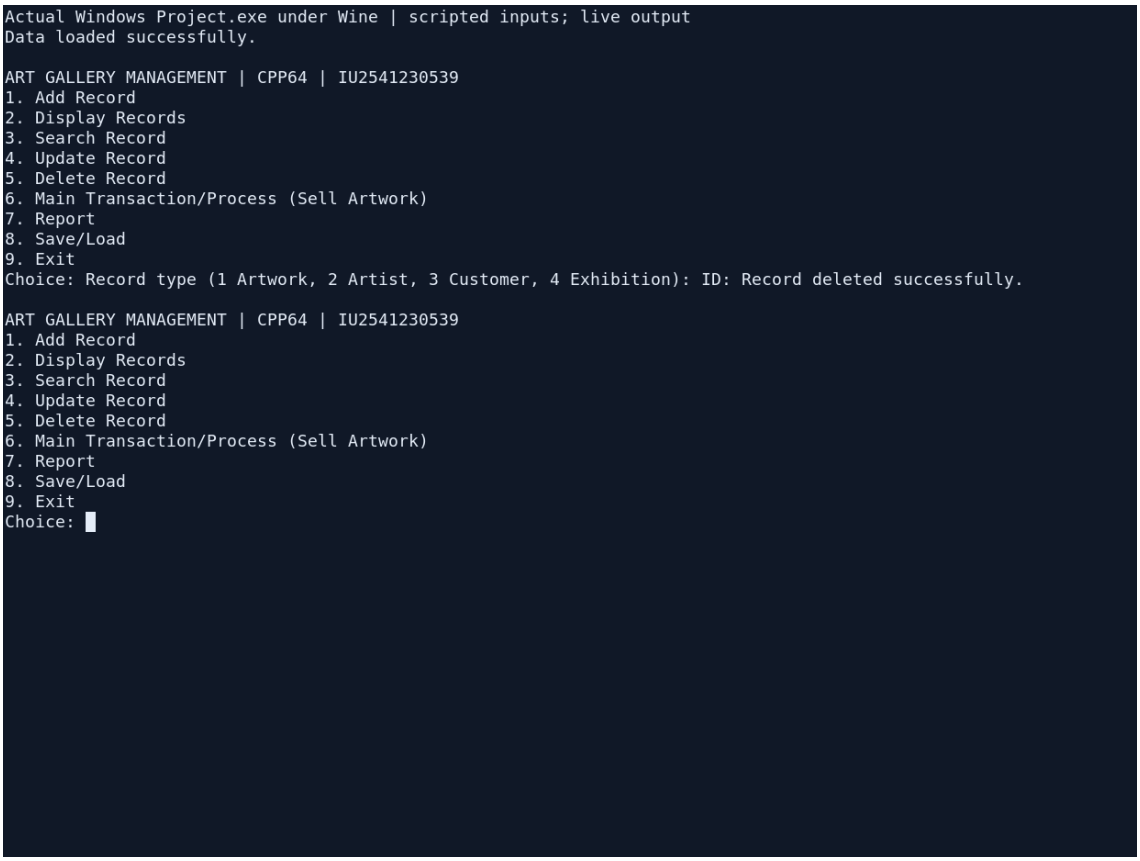
04_Search.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Record type (1 Artwork, 2 Artist, 3 Customer, 4 Exhibition): ID: New title: Artist ID: Price (INR): Reco
rd updated successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

05_Update.png



06_Delete.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Artwork ID: Customer ID: Artwork #101 | Monsoon Valley | Artist #1 | INR 25000.00 | AVAILABLE
Confirm purchase (YES): Data saved successfully.
Sale completed. Transaction #1 | INR 25000.00

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

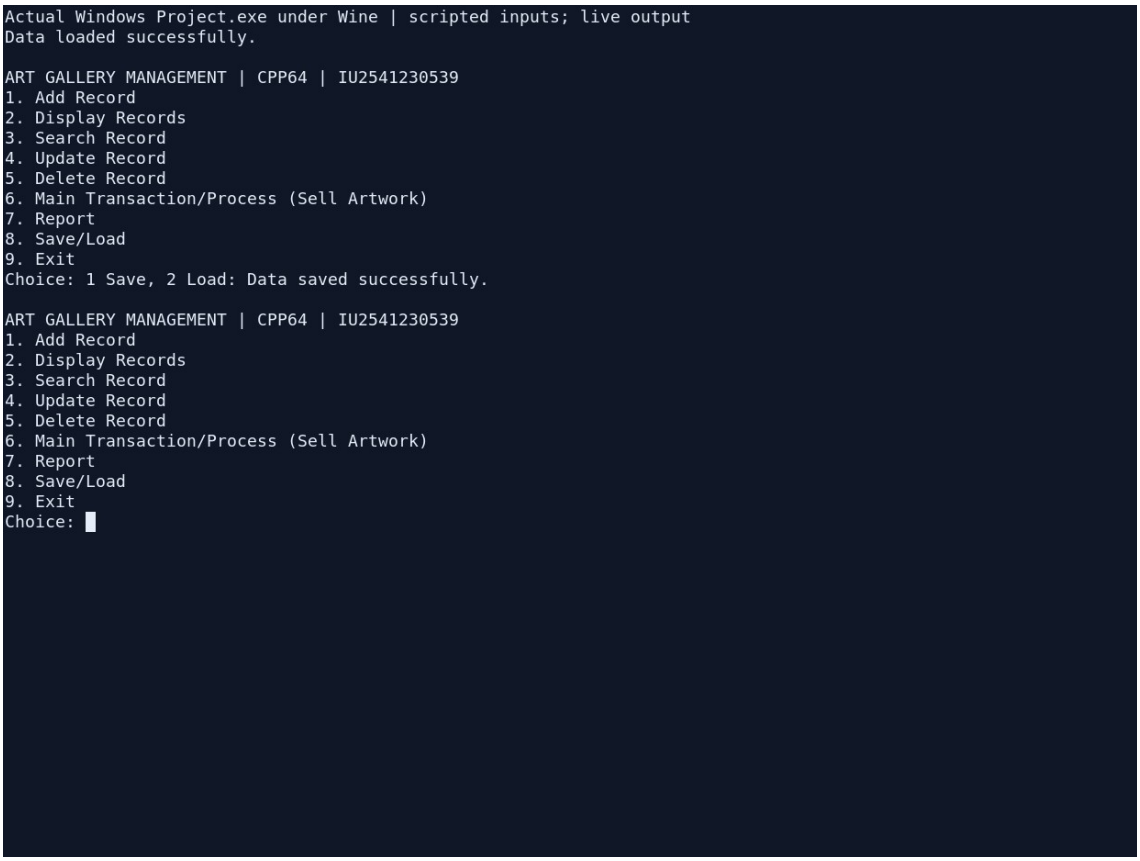
07_MainTransaction.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: GALLERY REPORT
Artworks: 5 | Available: 5 | Sold: 0
Artists: 3 | Customers: 3 | Exhibitions: 2
Available inventory value: INR 144000.00
Sales revenue: INR 0.00

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

08_Report.png



09_Save.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: 1 Save, 2 Load: Reload discards unsaved changes. Type YES: Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

10_Load.png

```
Actual Windows Project.exe under Wine | scripted inputs; live output
Data loaded successfully.

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice: Record type (1 Artwork, 2 Artist, 3 Customer, 4 Exhibition): ID: Artwork #106 | Persistent River | Artis
t #1 | INR 12345.67 | AVAILABLE

ART GALLERY MANAGEMENT | CPP64 | IU2541230539
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction/Process (Sell Artwork)
7. Report
8. Save/Load
9. Exit
Choice:
```

11_Persistence.png

11. Testing and Validation

A clean verification suite was prepared for the merged implementation. It targets the merged Linux build and covers loading, display, search, CRUD operations, sales, reference protection, reporting, save behavior and separate-process persistence. The Windows executable is intentionally not copied from the first project because that binary predates the merged data fields; the supplied build_windows.bat should be used to create the final Windows executable locally.

Test	Result
Compilation	Merged C++17 source compiled successfully with -Wall -Wextra -pedantic.
Load / Restart	Sample V2 data loaded successfully from an external data directory.
Display	Artwork records displayed with category and availability.
Search	Artwork ID lookup returned the expected record.
Sale	Artwork 101 was sold to customer 1 after YES confirmation.
Report	Report showed 5 artworks, 4 available, 1 sold and INR 25,000.00 revenue after the test sale.
Persistence	A separate process reopened the saved snapshot and reported artwork 101 as SOLD.
Legacy compatibility	FileManager retains support for the original V1 snapshot format.

The supplied first project also contains automated scenario transcripts, CSV/JSON results and sanitizer logs. Those artifacts remain in the final package as supporting evidence; they should be rerun after any further local code modification.

12. Challenges and Solutions

- Duplicate IDs → validated before insertion.
- Invalid numeric/text input → centralized validation helpers and exceptions.
- Artwork sold twice → sold state is checked before a sale.
- Deleting referenced artists/customers → relationship checks block destructive deletion.
- Unsaved data loss during reload → reload requires explicit YES confirmation.
- Corrupt persistence data → file format, counts, relationships and transaction state are validated before live state replacement.
- Partial save risk → temporary-file write followed by replacement and backup handling.
- Large artwork collections → dynamic array growth doubles capacity when needed.

13. Results and Discussion

The final application provides a coherent academic demonstration of object-oriented C++ programming while remaining practical enough for a gallery record workflow. The strongest technical features are the validated entity relationships, polymorphic Person hierarchy, dynamic memory management, transaction protection and versioned persistence.

The richer fields selected from the second project improve the domain model: artwork category makes inventory more meaningful, while exhibition start date, end date and theme make exhibition records more realistic. These

additions were integrated into the existing robust architecture rather than maintaining a second parallel data model.

14. Limitations and Future Scope

- Single-operator console interface; no authentication or role-based access.
- No online payment gateway, refunds or tax calculation.
- No multi-user concurrency or server database.
- No image management for artwork photographs.
- Future versions could use SQLite/MySQL, a GUI/web front end, user authentication, artwork image storage, advanced reports, search filters and exhibition scheduling conflict detection.

15. Conclusion

The Art Gallery Management system successfully demonstrates a modular C++17 application for maintaining gallery records and processing artwork sales. By combining the strongest qualities of both supplied projects, the final version offers stronger validation, safer persistence, richer gallery data, clearer reporting and better supporting documentation. The implementation is suitable as an academic Object-Oriented Programming project and is designed to be understandable during demonstration and viva.

16. References and Source Basis

The final project was constructed from the two uploaded project packages supplied for this task. No external factual source, statistic, author, publication date or URL has been fabricated. The technical claims in this report describe the implemented source code and included project artifacts.

- Project package 1 — Art Gallery Management / CPP64 / IU2541230539: source code, UML, screenshots, tests, report and README.
- Project package 2 — Art Gallery Management / CPP64 / IU2541230539: source code, data files, UML images, report and README.
- Actual external GitHub repository URL: to be added by the student only after the repository is genuinely created.

17. Appendix: Viva / Run Instructions

17.1 Windows Build

From the extracted project root, run build_windows.bat using a C++17 MinGW-w64 compiler. The script creates Executable\Project.exe.

17.2 Run

Double-click Run_Project.bat. The launcher passes the Data directory explicitly, so the project can be run from any extracted location.

17.3 Suggested Demonstration

- Choose 2 → 1 to display artworks.
- Choose 3 → 1 → 101 to search Monsoon Valley.

- Choose 6 → artwork 101 → customer 1 → YES to demonstrate a sale.
- Choose 7 to show the updated report.
- Exit with 9, restart the application and search artwork 101 to demonstrate persistence.

17.4 Viva Points

- Person is abstract so Artist and Customer can provide their own display implementations.
- Gallery owns the application records and coordinates business rules.
- FileManager separates persistence from business logic.
- Prices are stored as integer paise to avoid floating-point money calculations.
- Sold artwork is protected from modification and resale.
- Transactions are checked against artwork, customer and sold status during load.